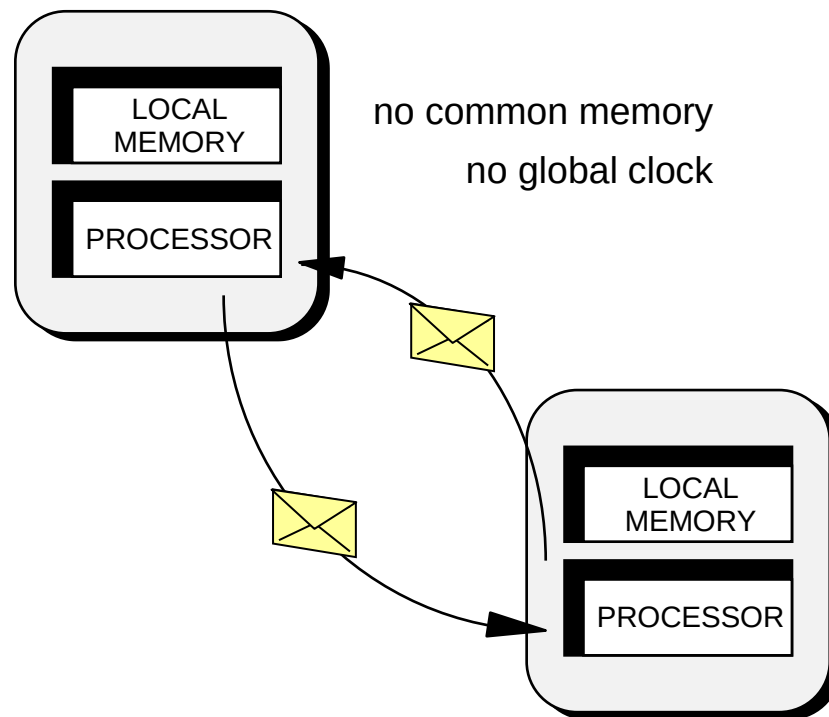


DISTRIBUTED SHARED MEMORY

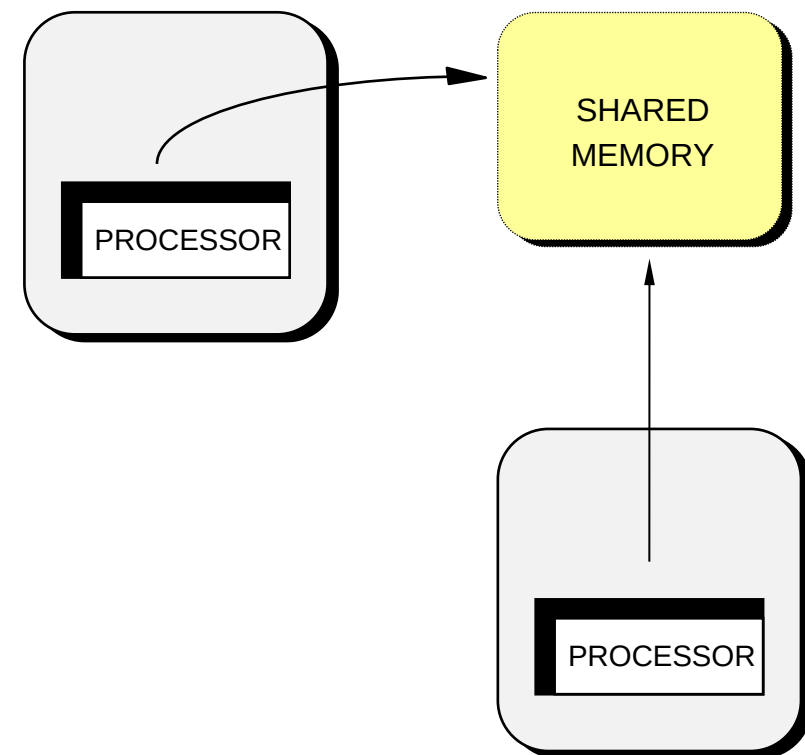
Message Passing

easy to implement
hard to program



Distributed Shared Memory

hard to implement
easy to program

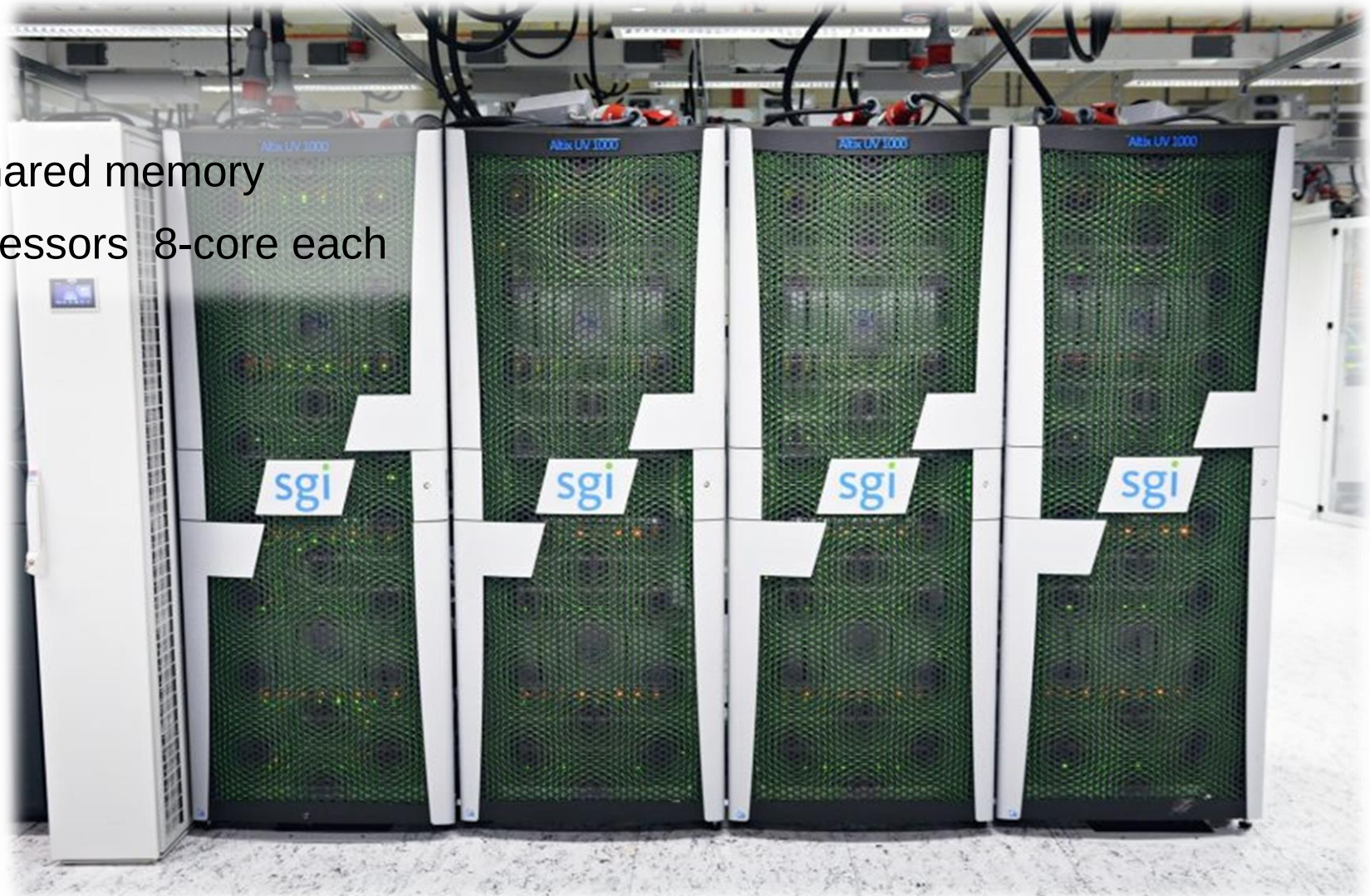


Chimera

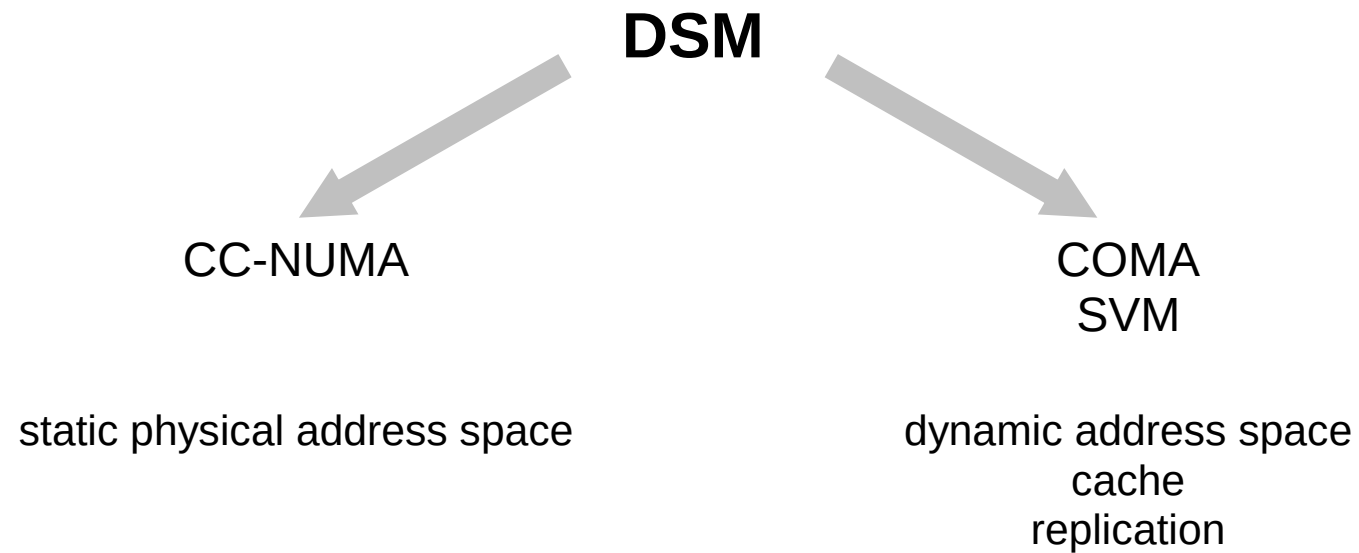
SMP

16 TB shared memory

256 processors 8-core each



DSM architectures



CC-NUMA = Cache Coherent Non Uniform Memory Access

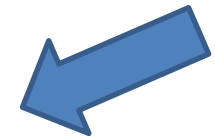
- provides a static physical address space
- each memory unit represents a static portion of the global address space

COMA = Cache Only Memory Architecture

- provides a dynamic address space
- the memory of a node is used as a large cache for the node's processor
- memory units are automatically **replicated** and migrated on demand
- coherence is maintained by a hardware **coherence protocol** (typically a directory-based **write/invalidate** protocol)

SVM = Shared Virtual Memory Systems

- implemented in software, memory unit: page / variable / object
- **replication** (*master replica, owner, CopySet – CS*)
- **write/invalidate** coherence protocol (update on *page miss / page fault*)
- any other strategies of updates (**write-update** coherence protocols)?



Hierarchical-directory coherence protocols:

- hierarchy used to locate memory units on memory misses
- directories at each level of the hierarchy maintain information about copies located in a sub-hierarchy

Flat-directory coherence protocols:

- preferable from a fault-tolerance point of view, as the failure of an intermediate node in the hierarchy could cause the loss of the whole underlying sub-hierarchy
- directory entries can be distributed among the nodes

Static distributed directory:

- *dynamic owner*, *static manager*

Dynamic distributed directory:

- *probable owner*



CHECKPOINTING IN DISTRIBUTED SYSTEMS (*RECALL*)

General approaches to store system state:

- log-based recovery
- state-based recovery

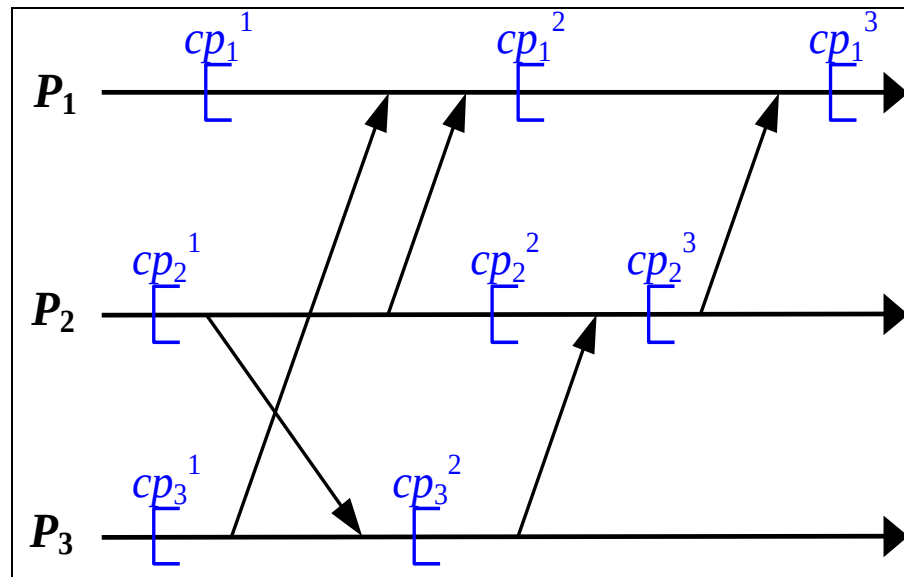
both require stable storage

Checkpointing:

- local checkpoint cp_i
- global checkpoint $CP = \langle cp_1, cp_2, \dots, cp_n \rangle$, cp_i : for each P_i
- only $CP^\bullet$ are of interest
- however, not necessarily $RL = CP^\bullet$ (not necessarily $rp_i = cp_i$)

Problems:

- recovery line consistency (orphan messages)
- domino effect, continuous restart (recovery livelock)



Major techniques:

- communication induced checkpointing
 - ineffective for general message-passing
 - quasi-synchronous works pretty well
- coordinated checkpointing
 - costly interprocess synchronization
- independent checkpointing
 - consistency problem (domino effect avoidance)
- hybrid = independent + occasional synchronization

Cost reduction:

- dependency tracking
- message semantics



RECOVERY IN DSM SYSTEMS

1. Recovery problem

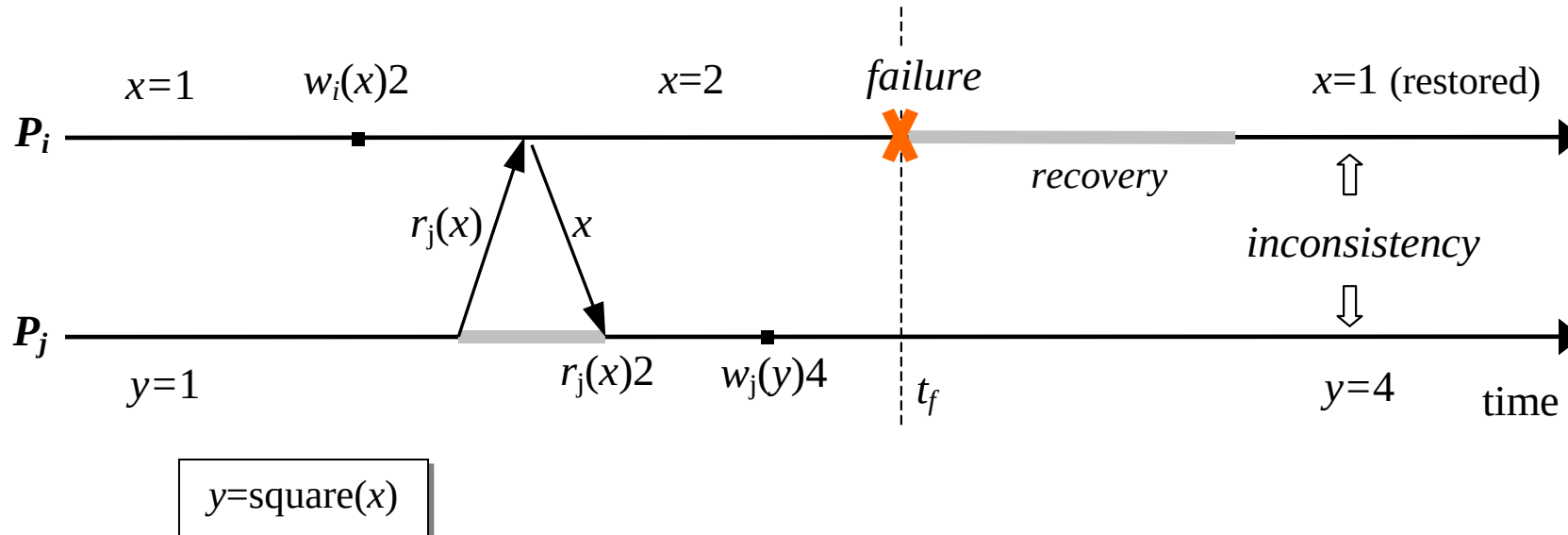
- node failures (whole content of the volatile memory used by P_i is lost) result potentially in unavailability of the lost shared objects
- restoring some predefined value of a lost object may violate the consistency of the whole shared memory space

2. Consistent recovery line

- process state → memory state
- process are irrelevant – all we care is the memory state (shared objects' values)
- local (process) checkpoint → object (memory) checkpoint
- the restored object's value must reflect the history of operations performed on the object by the coherence protocol

RECOVERY PROBLEM

- when process P_i fails, the whole content of the volatile memory used by P_i is lost, resulting in temporal unavailability of the lost objects
- if any of them was, at that moment, a unique replica of a given object, the DSM system will not be able to serve any further access issued to this object, until it is restored
- restoring the initial value of the lost object may result in memory inconsistency



Approaches

I. write-update coherency protocols

- no need for recovery (no additional protocol required)
- costly, write-broadcasts, full-replication (impractical)

II. write-invalidate coherency protocols

- (1) adaptation of message-passing recovery algorithms into a DSM environment
(= checkpointing the memory)
- (2) rollback dependency tracking based on semantic analysis of coherence messages (using message logging originally developed for message-passing)
- (3) checkpoint replication (extends the coherence protocol)



(1) Adaptation of message-passing recovery algorithms

- as each write operation performed by a process, changes the memory state (= process state), the DSM controllers save a **checkpoint (=copy) of a modified object** (replica) accordingly to some checkpointing protocol
e.g.:
 - enforcing a checkpoint before allowing other processes to read the modified object (***dirty page flush***)
- extends message-passing checkpointing algorithms with special data structures (conforming to DSM memory units) used to store checkpoints of DSM objects (e.g. *twin pages*), and with supplementary operations performed by DSM controllers (i.e. *owners* and *managers* of shared objects) ensuring correct maintenance of those structures in spite of failures

Full checkpointing

- a DSM node flushes the entire shared memory cache, i.e. all DSM objects cached locally

Incremental checkpointing

- a DSM node flushes only those objects that have been modified since the last checkpoint

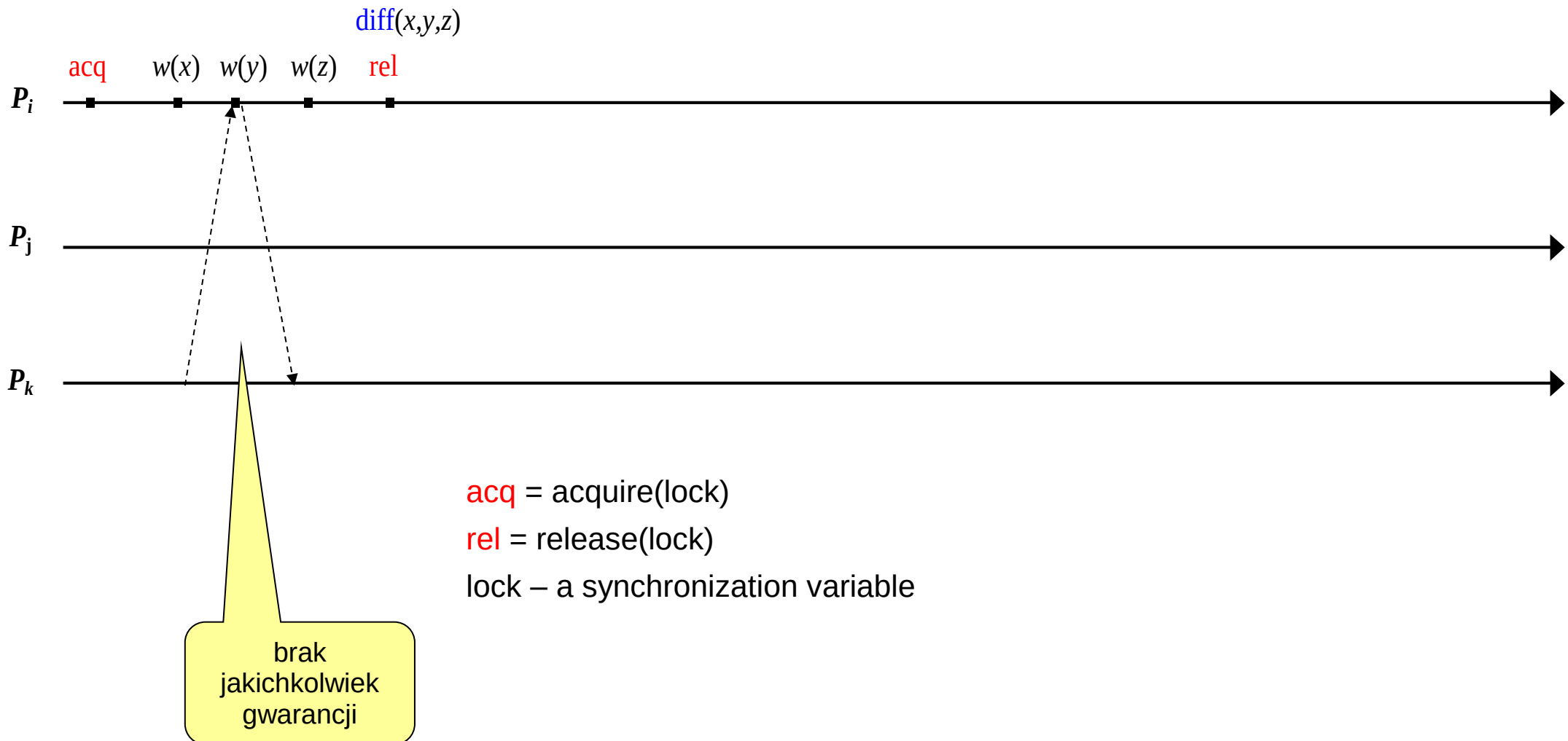
Differential checkpointing

- a new checkpoint adds only the actual modifications (***diffs***) of modified objects
- *diffs* – nothing new in page-based consistency protocols
- comfortable in variable/object-based DSM

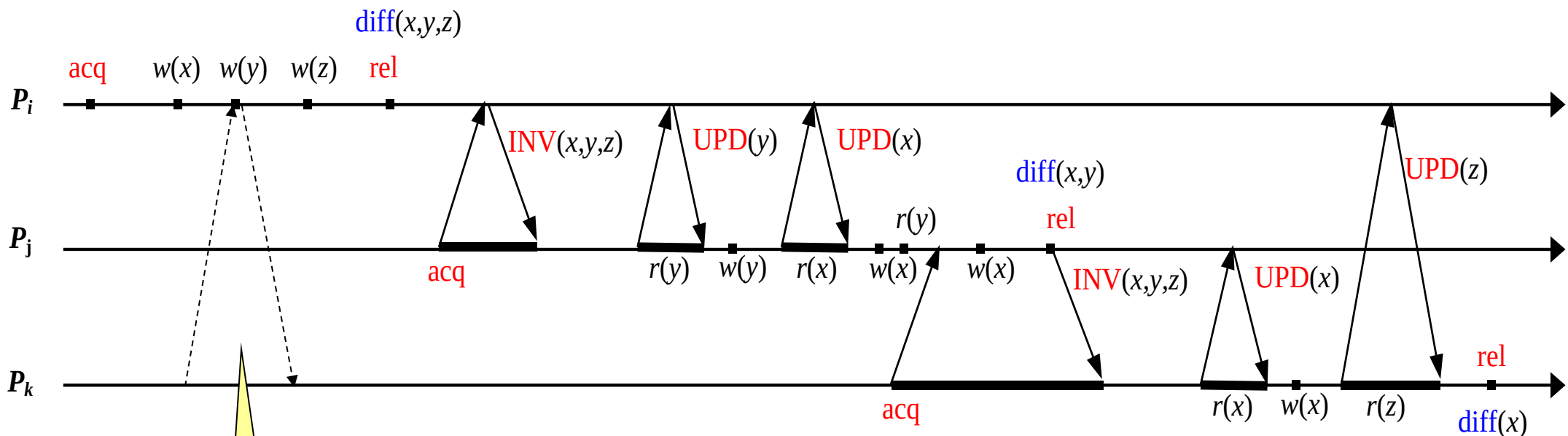
(2) Dependency tracking of the coherence protocol

- since the **coherence protocol** maintains the memory consistency by implicitly exchanging messages between nodes, the memory state of each process depends on the communication history pattern.
- **messages of a coherence protocol** are analyzed (and perhaps logged) in order to track dependencies between replicas' values
- **dependency tracking** can efficiently make use of message semantics: many messages of a coherence protocol do not change the memory state (e.g. read requests issued to the *owner*, owner localization requests, answers processed by the *manager*, etc.)
- dependency tracking can be tuned for a particular coherence protocol

Lazy Release Consistency (LRC) [1]



Lazy Release Consistency (LRC)

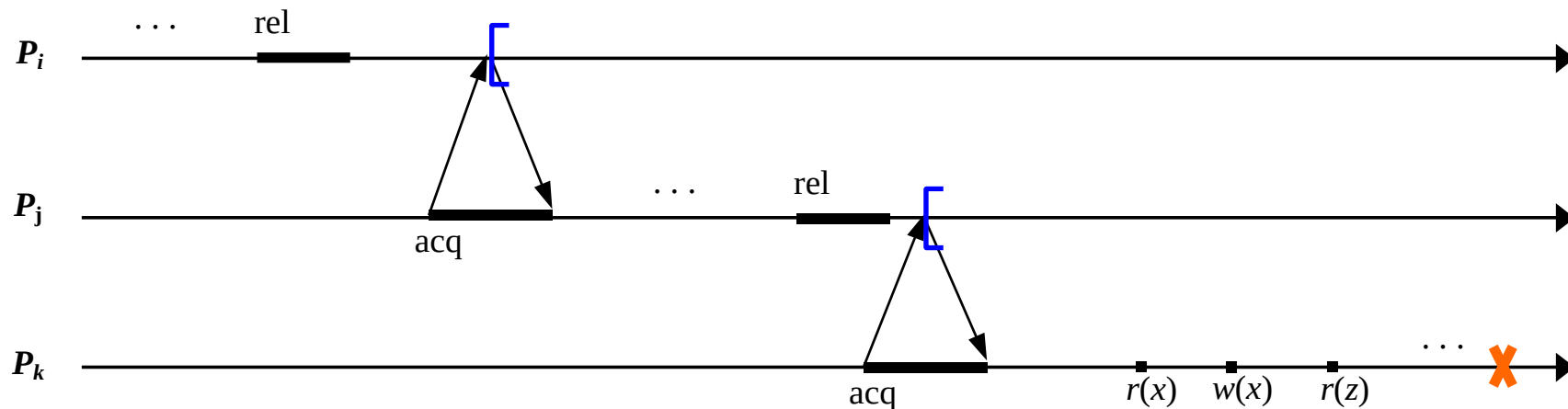


brak
jakichkolwiek
gwarancji

Checkpointing techniques

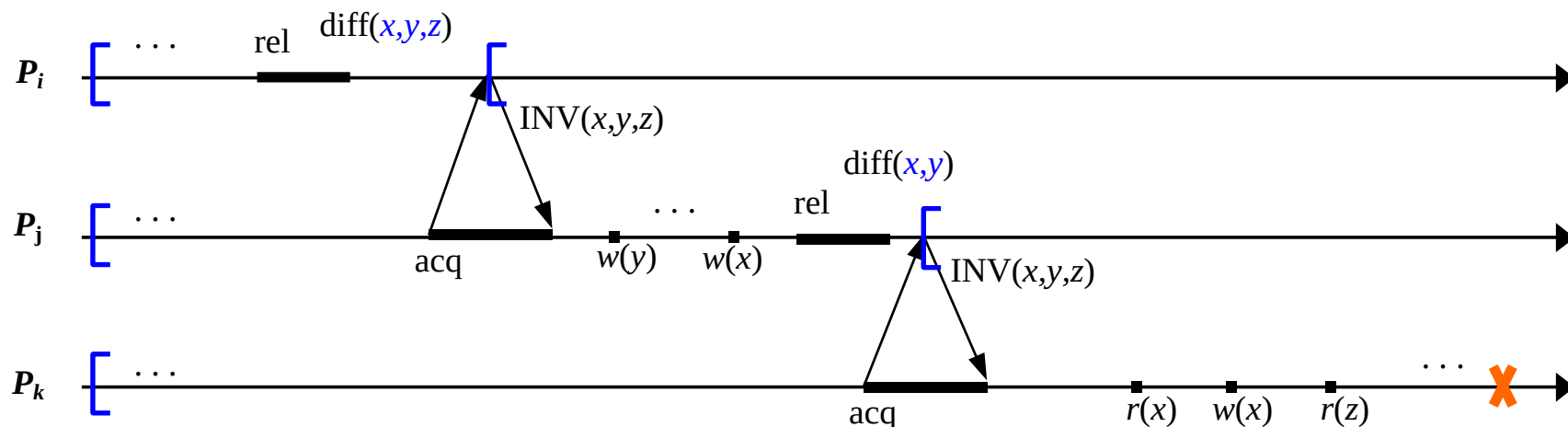
Janssens-Fuchs:

- communication-induced full checkpointing
- dependency tracking: dependencies are due to accesses to synchronization variables
- checkpointing on access to synchronization variables



Coherence-based Coordinated Checkpointing (CCC)

- incremental/differential checkpointing: the dirty-page lists give a summary of modifications needed for updating the memory
- much smaller checkpoints



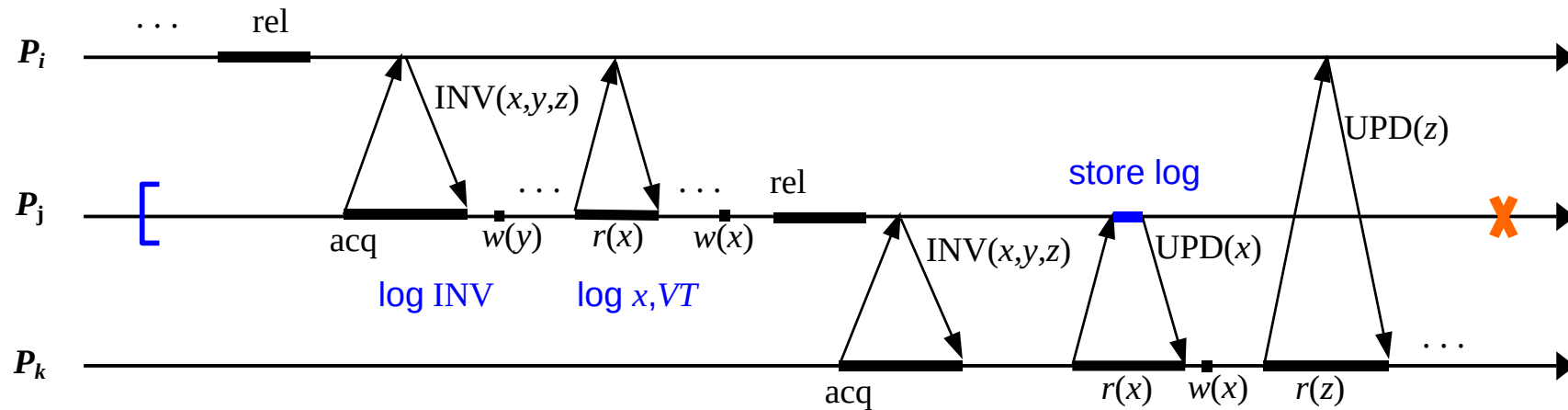
Logging techniques

Shared-Access Tracking (SAT)

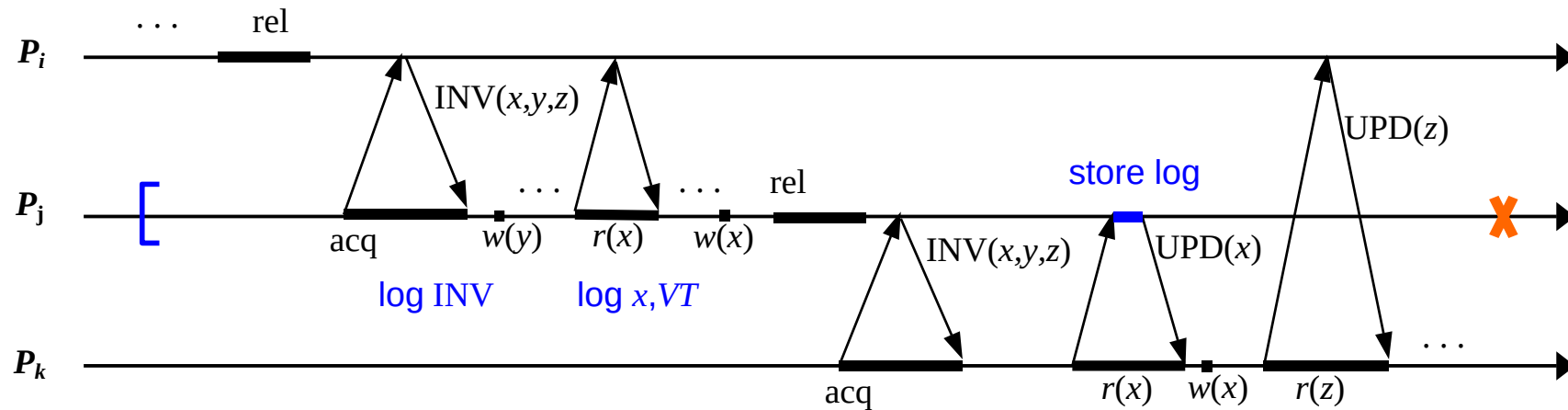
- basic idea: all accessed memory objects must be logged before they are read; on recovery all memory accesses (being nondeterministic) can be replayed with the same data at the same access points
- vector clock keeps track of the access points
- optimistic logging: **volatile** log of two types of information: (1) INV messages (dirty-page lists) received during acquire, and (2) UPD messages received on access misses
- periodical checkpointing (independent) limits log size and speeds up the recovery

LAZY RELEASE CONSISTENCY

(6)



- volatile log is flushed to **stable** storage before any modified object (*dirty*) is transferred to another process (i.e. before sending UPD)



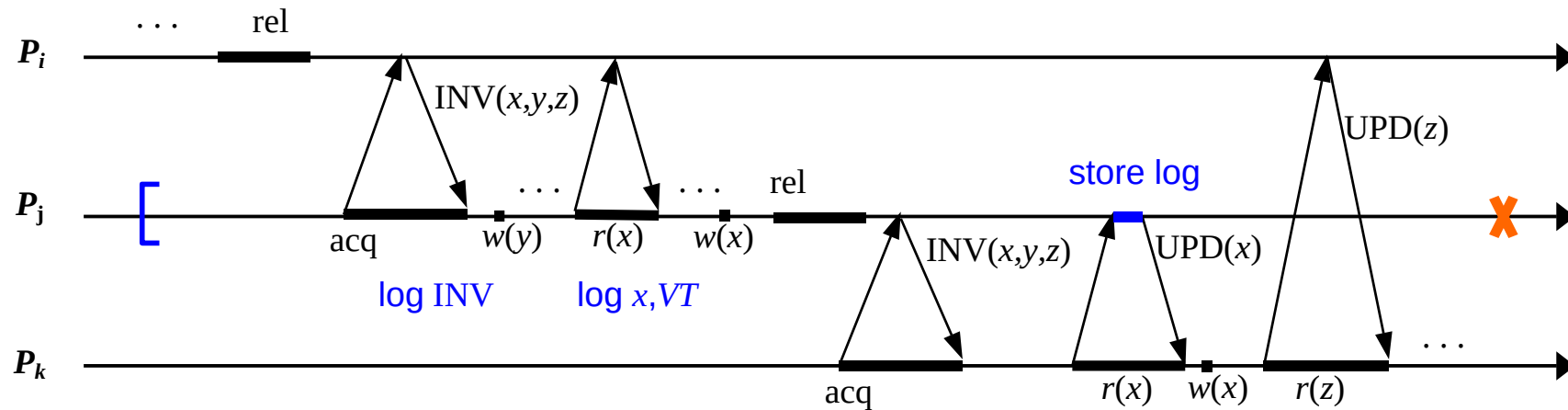
Recovery:

- on recovery, the process is rolled back to the most recent checkpoint and replays the log
- this log contains lists of dirty objects to be **invalidated** on each synchronization point
- **updating** the invalidated pages from the log is performed on access misses

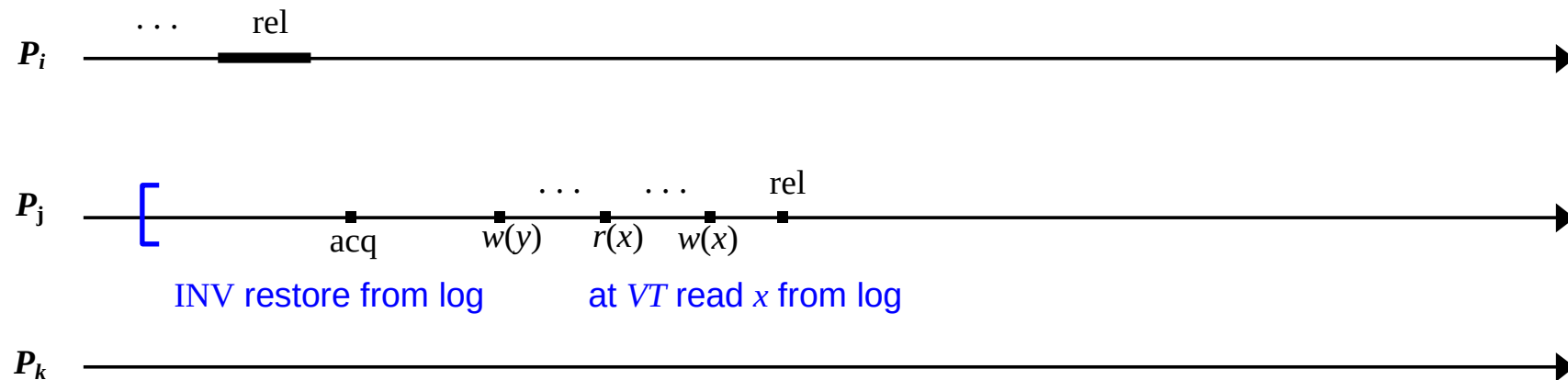


LAZY RELEASE CONSISTENCY

(8)



Recovery:



(3) Checkpoint replication

- replication can help to restore the state of a DSM node on recovery – the recovery protocol can fetch the values of all restored objects from available replicas on other nodes (preserving consistency of the whole DSM memory)
- in the best possible solution, the recovery demands no additional replication as the replicas maintained by the coherence protocols are sufficient to ensure recovery

example: A full-replication system using write-update protocol – every DSM node holds a replica of each shared object, atomically updated at every modification.

Although this solution is very simple and is N -resilient, it suffers from a very large update cost in the coherence protocol.

(3) Checkpoint replication (continued)

- checkpoints are stored not on *stable storage* but in the DSM memory itself
- the recovery protocol backups each checkpoint at distinct sites
- checkpoints can be replicated accordingly to the required level of the *failure resilience*

Atomic Consistency model

Let H_i denote the set of all operations issued by P_i and H denote the set of all operations issued by the system ($H = \bigcup_i H_i$). Let $o1 \rightarrow_{RT} o2$ mean that operation $o1$ finishes in real time before $o2$ starts.

An execution of DSM operations is atomically consistent if there exists a serialization $\mapsto$ of the operations, such that:

$$\forall_{o1, o2 \in H} o1 \rightarrow_{RT} o2 \Rightarrow o1 \mapsto o2$$

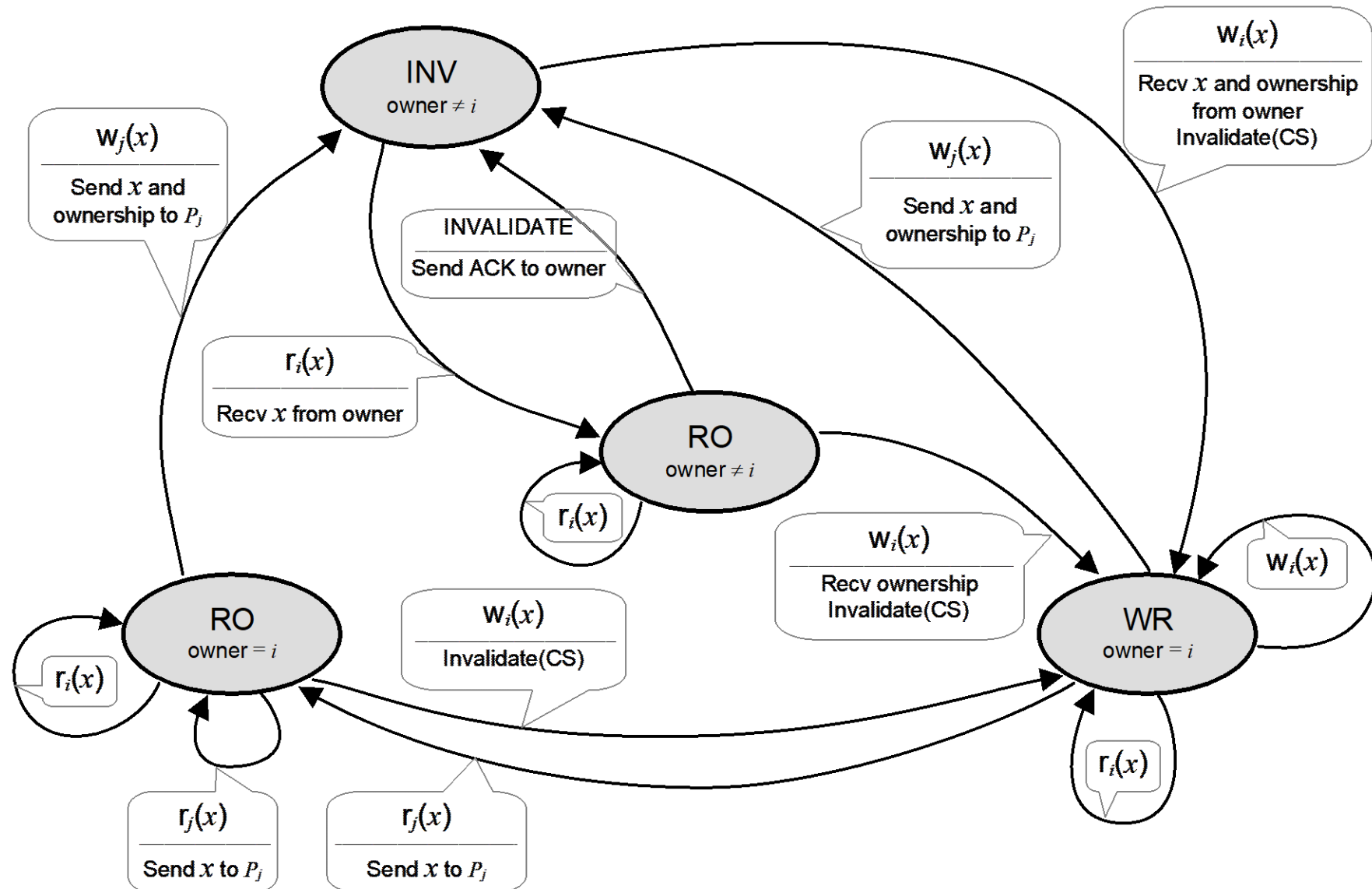
satisfying two conditions:

AC1 (legality): $\forall_{w(x)v, r(x)v \in H} \exists_{o1(x)u \in H} :: u \neq v \wedge w(x)v \mapsto o1(x)u \wedge o1(x)u \mapsto r(x)v$

AC2 (ordered writing): $\forall_{w^1(x), w^2(x) \in H} w^1(x) \mapsto w^2(x) \vee w^2(x) \mapsto w^1(x)$

Li-Hudak coherence protocol for atomic consistency [2]

- owner(x) holds master replica of x and a copyset $CS(x)$
- replica states:
 - WR (*writeable*) – exclusive master replica
 - RO (*read-only*) – shared replicas
 - INV (*invalid*) – object unavailable locally
- P_i reads $r_i(x)$ locally available replicas (RO, WR) instantaneously
- P_i writes $w_i(x)$ local WR replicas instantaneously
- before P_i accesses an unavailable object x (INV replica) it must get a replica with a current value of x (update)
- current value is the most recently written (in real time)
- before P_i writes to x it must become owner(x) and invalidate all replicas in $CS(x)$



Checkpoint-recovery protocol [4]

1. checkpoints are replicated
 - new replica states C (*checkpoint*) and ROC (*read-only checkpoint*)
 - CCS (*checkpoint copyset*) along with CS
2. we combine synchronization overhead of the coherence protocol (invalidations) with checkpointing overhead (checkpoint establishment)
 - coherency induced checkpoints
(try to reuse existing replicas from CS in CCS)
3. consistency of checkpoints
 - communication induced checkpoints
(taken each time coherency dependencies arise between processes)

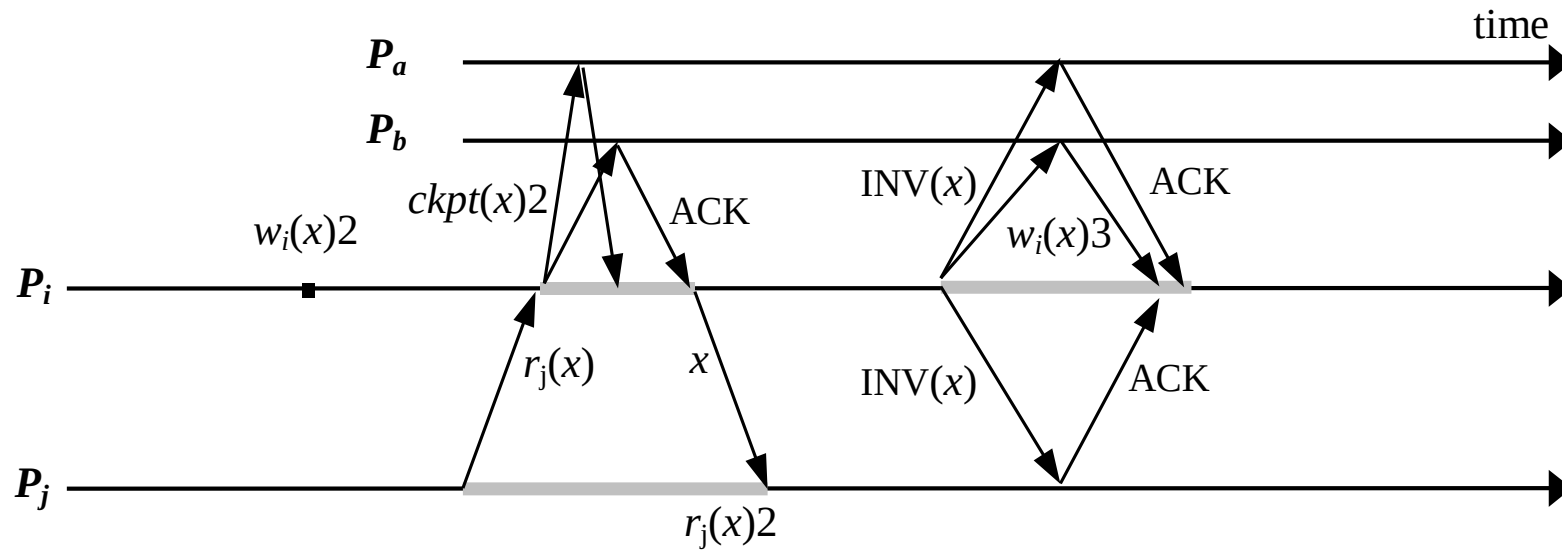
ROC (*read-only checkpoint*) replica of x :

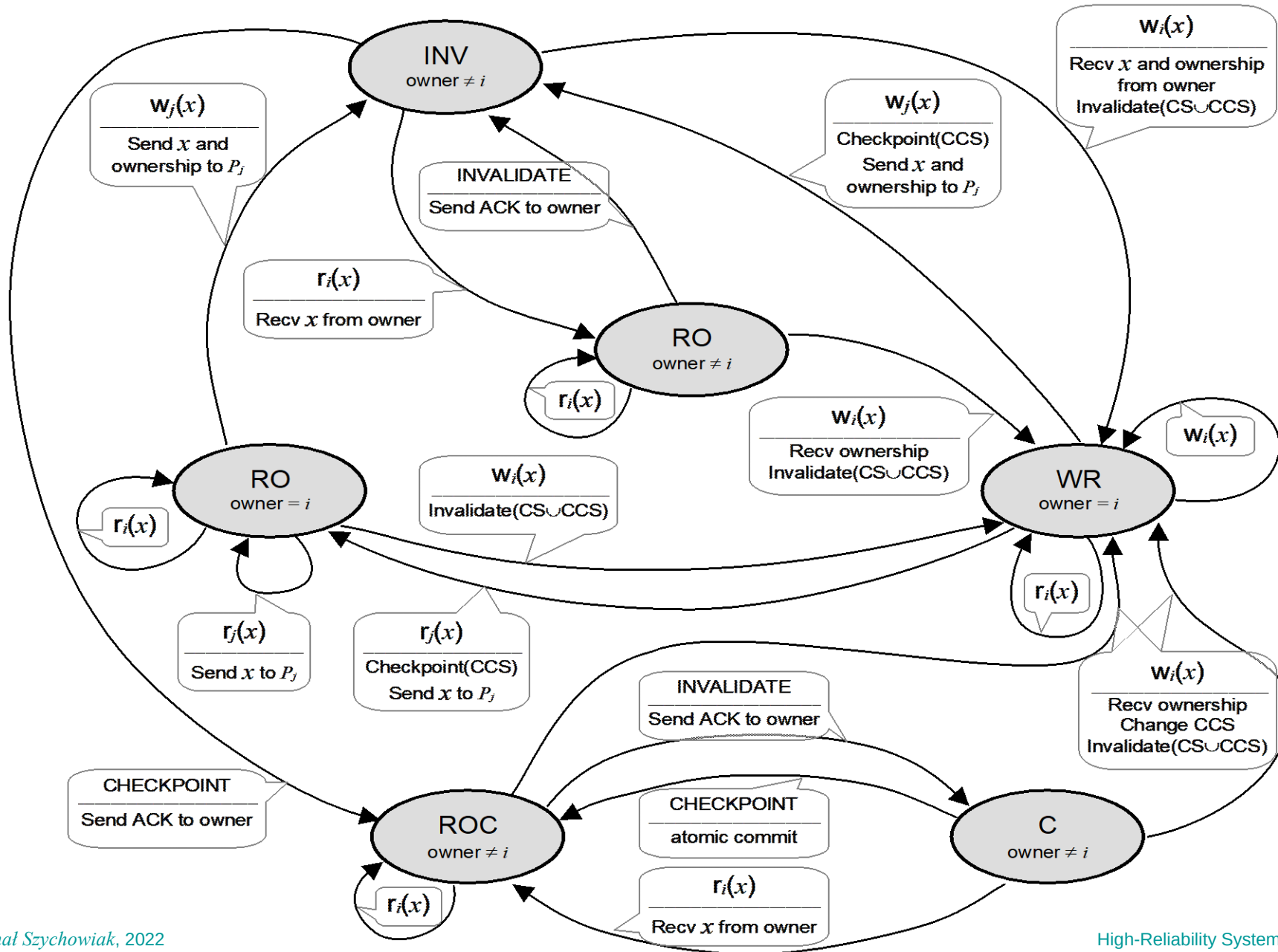
- its value reflects the state of x at the moment of the last checkpoint
- as since then, no further write was performed on this object, a ROC replica reflects the current value of x .
- therefore, on any read access to x , a ROC replica is used as if it was in RO state
- independently of ROC replicas there can be several RO replicas of x maintained as in the original coherency protocol.
- all checkpoint replicas are in state ROC directly after the checkpointing operation, until some process issues a write request to x .
- at that moment ROC replica switches into state C (recall, that RO replicas are invalidated then)

C (*checkpoint*) replica of x :

- not available for read nor write operations on x (available only on recovery to restore a consistent state of DSM)

Checkpointing example:

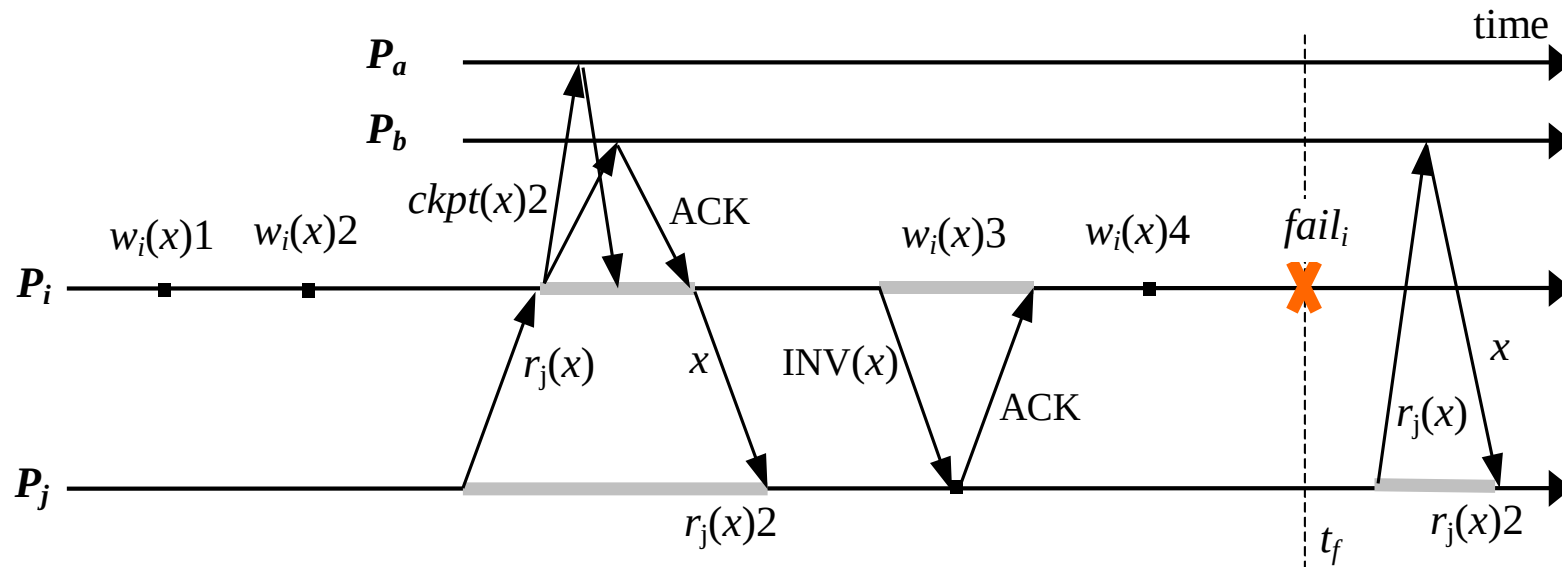




Object prefetching

- the extended protocol offers the benefit of prefetching object values in result of checkpointing (= updating ROC replicas)
- this can reduce the overall cost of the coherency operations, eliminating the read fault for checkpoint replicas (ROC)
- therefore no communication cost ever incurs on the read access issued to a checkpoint replica (ROC)
- note, that the prefetch does not increase the cost of the coherency operations, since no unnecessary local invalidation is performed due to the update of a checkpoint replica

Delayed checkpointing:



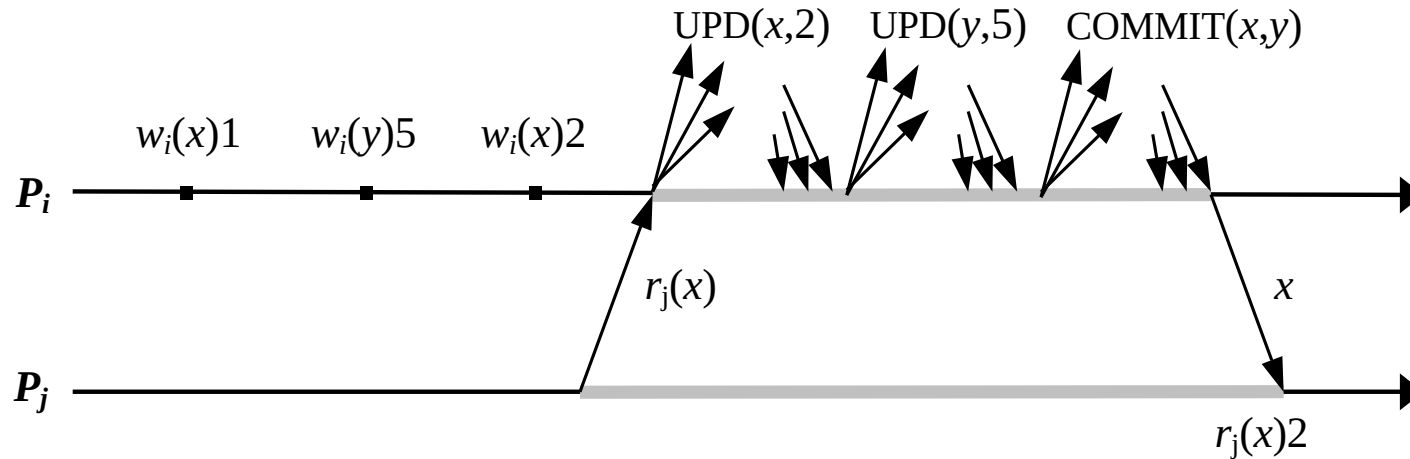


Checkpoint atomicity

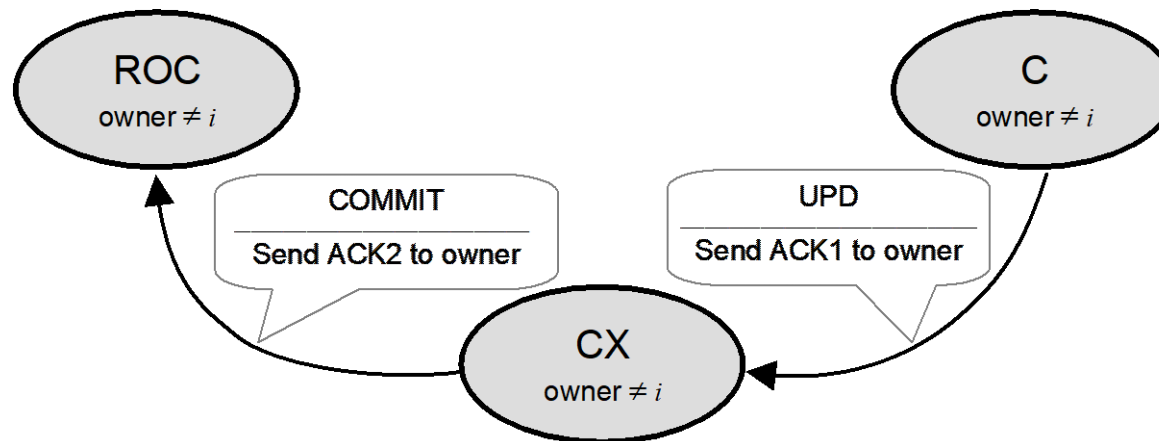
Remarks:

- at the moment of checkpointing x , P_i can also own some other object y
- if P_i would fail after checkpointing x but before checkpointing y , the atomic consistency of the memory could be violated on recovery – since the formerly checkpointed value of y will be inconsistent with checkpointed value of x
(if the recovered state of the memory reflects modifications performed by P_i on x , it has to reflect also those done on y)
- therefore, at any object's ownership transfer, P_i is required to checkpoint all owned objects not yet checkpointed

Burst checkpointing



Atomicity of burst checkpointing



Optimizations:

- boundary restriction:

$$f \leq \boxed{nr_{min} \leq |CCS(x)| \leq nr_{max}} \leq n$$

- correlation between $CCS(x)$ and $CS(x)$:

$$|CS(x)| \geq nr_{min} \Rightarrow CCS(x) \subseteq CS(x)$$

$$|CS(x)| < nr_{min} \Rightarrow CS(x) \subseteq CCS(x)$$

Partitioning:

- $f < \left\lceil \frac{n}{2} \right\rceil$
- correct processes are in a majority partition
- only objects in this partition can evolve further

- [1]  Peter Keleher, Alan L. Cox, and Willy Zwaenepoel, “Lazy Release Consistency for Software Distributed Shared Memory”, *19th Annual International Symposium on Computer Architecture*, 1992.
- [2]  K. Li and P. Hudak: “Memory Coherence in Shared Virtual Memory Systems”, *ACM Transactions on Computer Systems*, 24(8), 1989.
- [3]  Jerzy Brzeziński, Michał Szychowiak: Reliability of Distributed Shared Memory Systems, *European Conference on Research and development for Information Society (ISTHmus 2000)*, April 2000.
<https://www.cs.put.poznan.pl/mszychowiak/publications/ISTHmus2000-rDSM.pdf>
- [4]  Jerzy Brzeziński, Michał Szychowiak: Replication of Checkpoints in Recoverable DSM Systems, *21st Parallel and Distributed Computing and Networks (PDCN 2003)*, February 2003
https://www.cs.put.poznan.pl/mszychowiak/publications/Atomic-rDSM_IASTED.pdf